

GC 비교 테스트 기록

0. 목차

- [1. 테스트 목적](#)
- [2. 사용 도구 및 환경](#)
- [3. 테스트 과정 및 환경 변화](#)
- [4. 환경 변화에 따른 결과 비교](#)
- [5. 최종 결론](#)
- [6. 재현 방법 \(Step-by-step\)](#)
- [7. GCeasy 분석 방법](#)

해당 테스트는 Claude Code 와 협업하여 진행하였습니다.

1. 테스트 목적

배경

Spring Boot 애플리케이션을 실제 서버(Windows 11, Intel 9th i5, RAM 16GB)에 배포할 계획이다.

동일 서버에 nginx, MySQL, Spring Boot를 함께 운영해야 하는 환경이었기 때문에, JVM의 GC 방식 선택이 전체 서버 안정성에 영향을 미칠 수 있었다.

이론상만 보면 GC보다 ZGC가 좋아보이지만 과연 어느 환경에서도 항상 좋은 결과를 낼까? 라는 의문에서 해당 테스트를 시작하게되었다.

GC 선택의 중요성

JVM은 힙 메모리에 쌓인 불필요한 객체를 정리할 때 애플리케이션을 잠깐 멈추는 STW(Stop-The-World) 이벤트를 발생시킨다.

이 멈춤 시간(pause time)이 길어지면 사용자 입장에서 응답 지연으로 체감된다.

Java 21 기준으로 대표적인 GC는 두 가지다.

GC	특징
G1 GC	Java 9 이후 기본 GC. 안정적인 처리량과 적절한 pause time의 균형
ZGC	초저지연(Low Latency) 특화. 힙이 충분할 때 pause time을 1ms 이하로 유지

테스트 목표

- G1 GC와 ZGC의 성능 차이를 실측 데이터로 확인한다.
- 힙 크기 설정이 각 GC 성능에 미치는 영향을 파악한다.
- 실제 서버 환경에 적합한 GC와 힙 설정을 도출한다.

2. 사용 도구 및 환경

측정 환경

항목	값
테스트 머신	MacBook pro (로컬)
Java 버전	Java 21
Spring Boot 버전	4.0.2
서버 포트	4201
테스트 엔드포인트	GET /api/v1/organization/filter-list
부하 도구	hey v0.1.5
부하 설정	-z 30s -c 10 (30초, 동시접속 10)

사용 도구

JVM GC 로깅 옵션

JVM 자체 기능을 통해 GC 이벤트마다 로그를 파일로 기록한다.

```
-Xlog:gc*:file=logs/gc-{gc-name}.log:time,uptime:filecount=5,filesize=20m
```

- gc* : 모든 GC 관련 이벤트 기록
- time,uptime : 실제 시각과 JVM 기동 후 경과 시간 함께 기록
- filecount=5 : 최대 5개 파일로 롤링
- filesize=20m : 파일당 최대 20MB

hey (부하 테스트 도구)

```
hey -z 30s -c 10 http://localhost:4201/api/v1/organization/filter-list``
```

- -z 30s : 30초 동안 지속적으로 요청
- -c 10 : 10개의 동시 접속 유지
- 출력 지표 : Requests/sec(초당처리량), Average latency(평균응답시간), P99 latency(전체 요청의 99%가 이 시간안에 처리), Max latency(최대응답시간)

GCEasy

- 주소 : <https://gceasy.io>
 - GC 로그 파일을 업로드하면 자동으로 파싱하여 시각화
 - 확인 지표
 - Max pause time(최대정지시간) ,
 - Avg pause time(평균정지시간) ,
 - GC throughput(GC 처리량) ,
 - 힙 사용량 ,
 - 객체 생성률
-

3. 테스트 과정 및 환경 변화

테스트는 총 3단계에 걸쳐 힙 설정을 조정하며 진행되었다.

1단계: 힙 크기 미지정 (JVM 기본값)

목적 : GC 특성의 기본 차이를 확인한다.

JVM은 `-Xmx` 를 지정하지 않으면 시스템 RAM의 약 25%를 최대 힙으로 자동 설정한다.
테스트 머신의 RAM이 충분했기 때문에 ZGC는 약 11GB, G1은 약 1GB 미만의 힙을 사용했다.

실행 명령어

```
# G1 GC
java \
  -XX:+UseG1GC \ -Xlog:gc*:file=logs/gc-
g1.log:time,uptime:filecount=5,filesize=20m \ -jar app.jar --
spring.profiles.active=dev-db
# ZGC
java \
  -XX:+UseZGC \ -Xlog:gc*:file=logs/gc-
zgc.log:time,uptime:filecount=5,filesize=20m \ -jar app.jar --
spring.profiles.active=dev-db
```

문제점

두 GC가 전혀 다른 메모리 환경에서 동작했다. ZGC는 11GB 힙을 사용했고 G1은 1GB 미만을 사용해 동일 조건의 비교가 아니었다. ZGC는 구조상 동시 작업을 위해 빈 공간을 활용하기 때문에 메모리를 더 많이 점유하는 경향이 있다.

2단계: 힙 크기 512m로 통일

목적 : 동일한 메모리 조건에서 GC 알고리즘 자체의 차이를 비교한다.

```
java -XX:+UseG1GC -Xms256m -Xmx512m ... java -XX:+UseZGC -Xms256m -Xmx512m ...
```

문제점

512MB는 ZGC가 동시 GC 작업을 수행하기에 지나치게 좁은 공간이다. ZGC는 살아있는 객체와 새로 할당되는 객체를 동시에 수용하면서 백그라운드로 GC를 수행하는데, 힙이 너무 작으면 여유 공간이 부족해 GC가 따라가지 못하는 상황이 발생한다. 실제로 한 차례 측정에서 ZGC의 Max latency가 247ms로 급등하는 스파이크가 관찰되었다.

이 단계는 극단적으로 작은 힙을 설정한 경우로, 실제 운영 환경을 반영하지 못한다는 한계가 있었다.

3단계: 실제 서버 환경 기준 힙 설정 (최종)

목적 : 실제 배포 서버(16GB RAM) 환경에 맞는 설정으로 비교한다.

배포 서버 환경을 기준으로 메모리 배분을 설계했다.

프로세스	할당
Windows 11 OS	약 3~4GB
nginx	약 200MB
MySQL	약 4GB
Spring Boot	약 6GB (힙 4GB + 비힙 1GB)
여유 버퍼	약 2~3GB

Spring Boot에는 `-Xms2g -Xmx4g` 를 적용했다.

- `-Xms2g` : 서버 기동 시 힙을 절반 확보하여 초반 힙 리사이징으로 인한 GC 이벤트를 줄임
- `-Xmx4g` : 최대 4GB 힙. ZGC가 동시 GC 작업을 수행하기에 충분한 여유 공간
- ZGC에는 Java 21에서 추가된 `ZGenerational` 옵션을 함께 적용

```
# G1 GC
java \
  -XX:+UseG1GC \ -Xms2g -Xmx4g \ -Xlog:gc*:file=logs/gc-
g1.log:time,uptime:filecount=5,filesize=20m \ -jar app.jar --
spring.profiles.active=dev-db
# ZGC
java \
  -XX:+UseZGC \ -XX:+ZGenerational \ -Xms2g -Xmx4g \ -
Xlog:gc*:file=logs/gc-zgc.log:time,uptime:filecount=5,filesize=20m \ -jar
app.jar --spring.profiles.active=dev-db
```

4. 환경 변화에 따른 결과 비교

hey 부하 테스트 결과

단계	힉 설정	GC	Req/s	Avg	P99	Max
1단계	미지정	G1	45,968	0.3ms	0.7ms	20.3ms
1단계	미지정	ZGC	37,897	0.3ms	1.0ms	35.9ms
2단계	512m	G1	45,250	0.3ms	0.8ms	14.9ms
2단계	512m	ZGC	37,661	0.3ms	1.2ms	247ms
2단계 (재측정)	512m	G1	44,650	0.3ms	0.9ms	20.1ms
2단계 (재측정)	512m	ZGC	40,523	0.3ms	1.0ms	34.0ms
3단계	2g~4g	G1	39,877	0.3ms	0.9ms	54.9ms
3단계	2g~4g	ZGC	45,032	0.3ms	0.7ms	18.9ms

단계별 해석

1단계 (힉 미지정)

ZGC가 11GB 힉을 사용하면서 매우 안정적인 pause time을 보였다. 그러나 G1은 1GB 미만의 힉에서 동작했으므로 공정한 비교가 아니다. 이 결과만으로 GC 우열을 판단할 수 없다.

2단계 (힉 512m)

동일 조건이 되자 G1이 안정적인 모습을 보였고, ZGC는 한 차례 247ms 스파이크가 발생했다. 512MB 힉은 ZGC의 동시 GC 메커니즘이 제대로 작동하기에 너무 작은 크기다. ZGC는 메모리가 부족하면 오히려 더 긴 pause time을 유발한다는 것을 확인했다.

3단계 (힉 2g~4g, 실제 서버 기준)

힉을 실제 운영 환경에 맞게 설정하자 결과가 역전되었다.

ZGC가 처리량(45,032 req/s)과 Max latency(18.9ms) 모두에서 G1을 앞섰다.

G1의 Max latency는 54.9ms로 오히려 ZGC보다 높게 나타났다.

이는 힉이 넉넉해지면서 ZGC가 설계대로 동시 GC를 수행한 결과다.

ZGC는 힉 크기에 민감하다. 힉이 작으면 G1보다 불안정하고, 힉이 충분하면 G1보다 더 낮은 pause time을 달성한다. 따라서 ZGC 도입 여부는 서버에 할당 가능한 메모리 규모를 먼저 파악한 후 결정해야 한다.

5. 결론

실제 서버(16GB RAM) 기준 권장 GC

ZGC 도입 권장

- 힙 4GB가 보장되는 환경에서 ZGC는 GC pause로 인한 응답 지연이 사실상 없음
- 50명 규모의 사용자 트래픽에서 힙 4GB는 충분한 여유 공간
- Java 21의 ZGenerational ZGC는 기존 ZGC 대비 처리량도 개선됨

메모리 환경별 GC 선택 기준

서버 메모리	Spring Boot 힙	권장 GC
4GB 이하	1GB 이하	G1 GC
8GB	2~3GB	G1 GC 또는 ZGC
16GB 이상	4GB 이상	ZGC

6. 재현 방법 (Step-by-step)

사전 준비

```
# hey 설치 (Mac)
brew install hey

# JAR 빌드
./gradlew bootJar

# 빌드 결과 확인
ls build/libs/*.jar
```

G1 GC 측정

```
# 1. 앱 백그라운드 실행
set -a && source .env && set +a && \
java \
  -XX:+UseG1GC \ -Xms2g -Xmx4g \ "-Xlog:gc*:file=logs/gc-
g1.log:time,uptime:filecount=5,filesize=20m" \ -jar build/libs/YOUR-
APP.jar \ --spring.profiles.active=dev-db > logs/app-g1.log 2>&1 &

# PID 기록
echo "PID: $!"

# 2. 앱 기동 확인
curl -s -o /dev/null -w "%{http_code}"
http://localhost:4201/api/v1/organization/filter-list
```

```
# 3. 부하 테스트
hey -z 30s -c 10 http://localhost:4201/api/v1/organization/filter-list

# 4. 앱 종료
kill {PID}
```

ZGC 측정

```
# 1. 앱 백그라운드 실행
set -a && source .env && set +a && \
java \
    -XX:+UseZGC \ -XX:+ZGenerational \ -Xms2g -Xmx4g \ "-
Xlog:gc*:file=logs/gc-zgc.log:time,uptime:filecount=5,filesize=20m" \ -
jar build/libs/YOUR-APP.jar \ --spring.profiles.active=dev-db > logs/app-
zgc.log 2>&1 &
# 2~4 동일하게 반복
```

주의사항

- G1 측정 종료 후 최소 3초 대기하고 ZGC 측정을 시작한다. 포트가 해제되지 않은 상태에서 재실행하면 기동에 실패한다.
- 로그 파일은 매 실행 시 덮어쓰인다. 여러 회차의 결과를 보존하려면 파일명을 회차별로 다르게 지정한다.
 - 예: gc-g1-round1.log , gc-g1-round2.log
- 부하 테스트는 앱이 완전히 기동된 후 실행한다. JVM warm-up이 완료되지 않은 상태에서 부하를 주면 결과가 왜곡될 수 있다.

7. GCEasy 분석 방법

접속 및 업로드

1. <https://gceasy.io> 접속
2. Choose GC Log File 버튼 클릭
3. logs/gc-g1.log 업로드 후 분석
4. 결과 캡처
5. 동일하게 logs/gc-zgc.log 업로드 후 분석

용어정리

지표	의미
Requests/sec	초당 처리한 요청 수 (Throughput). 높을수록 좋음

지표	의미
Average latency	전체 요청의 평균 응답 시간. 이상치를 숨길 수 있으므로 단독 판단 금지
P99 latency	상위 1%를 제외한 최대 응답 시간. 사용자 체감 성능의 핵심 지표
Max latency	가장 느렸던 단일 요청의 응답 시간. 간헐적 스파이크 탐지용

Requests/sec

- 서버 처리량 확인
- TPS 증가 시 latency와 함께 확인 필요

P99 latency

- 실제 사용자 체감 성능 판단 핵심
- 평균 latency보다 중요

지표	의미
GC Throughput	GC에 사용되지 않은 시간 비율. 높을수록 좋음 (99% 이상 권장)
Avg GC Pause Time	평균 STW(Stop-The-World) 멈춤 시간. GC 발생 빈도와 비용의 평균값
Max GC Pause Time	최대 STW 멈춤 시간. 사용자가 체감하는 최악의 지연
Object Allocation Rate	초당 객체 생성량 (MB/s). 부하 테스트 강도 및 코드 효율성 지표
Heap Usage	힙 사용량 추이. 톱니파 형태면 정상, 우상향이면 메모리 누수 의심

GC Throughput

- GC가 애플리케이션 시간을 얼마나 잡아먹는지 확인
- 99% 이상이면 일반적으로 양호

Heap Usage

- 메모리 누수 여부 판단
- Full GC 이후에도 지속 상승 시 의심

이상 징후 예시

현상	의심 원인
P99만 급격히 증가	특정 쿼리, Lock 경합, 외부 API 지연
Avg는 정상인데 Max만 매우 큼	간헐적 GC, 네트워크 스파이크
GC Pause 증가	Heap 부족, 객체 생성 과다
Heap Usage 우상향	메모리 누수 가능성
Requests/sec 감소 + CPU 높음	병목 발생 (DB, GC, Thread Pool 등)

두 결과 비교 방법

GCEasy 무료 플랜에서는 동시 비교 기능이 제공되지 않는다. G1과 ZGC 각각 업로드 후 수치를 별도로 기록하여 직접 비교한다.
